

2.3 大语言模型结构

当前绝大多数大语言模型结构都采用了类似 GPT 架构，使用基于 Transformer 架构构造的仅由解码器组成的网络结构，采用自回归的方式构建语言模型。但是在位置编码、层归一化位置以及激活函数等细节上各有不同。文献 [5] 介绍了 GPT-3 模型的训练过程，包括模型架构、训练数据组成、训练过程以及评估方法。由于 GPT-3 并没有开放源代码，根据论文直接重现整个训练过程并不容易，因此文献 [31] 介绍了根据 GPT-3 的描述复现的过程，并构造开源了系统 OPT (Open Pre-trained Transformer Language Models)。Meta AI 也仿照 GPT-3 架构开源了 LLaMA 模型^[37]，公

开评测结果以及利用该模型进行有监督微调后的模型都有非常好的表现。由于自 GPT-3 模型之后，OpenAI 就不再开源也没有开源模型，因此并不清楚 ChatGPT 和 GPT-4 所采用的模型架构。

本节将以 LLaMA 模型为例，介绍大语言模型架构在 Transformer 原始结构上的改进，并介绍 Transformer 模型结构中空间和时间占比最大的注意力机制优化方法。

2.3.1 LLaMA 的模型结构

文献 [37] 介绍了 LLaMA 所采用的 Transformer 结构和细节，与在本章 2.2 节所介绍的 Transformer 架构不同的地方包括采用了前置层归一化 (Pre-normalization) 并使用 RMSNorm 归一化函数 (Normalizing Function)、激活函数更换为 SwiGLU，并使用了旋转位置嵌入 (RoP)，整体 Transformer 架构与 GPT-2 类似，如图2.4所示。

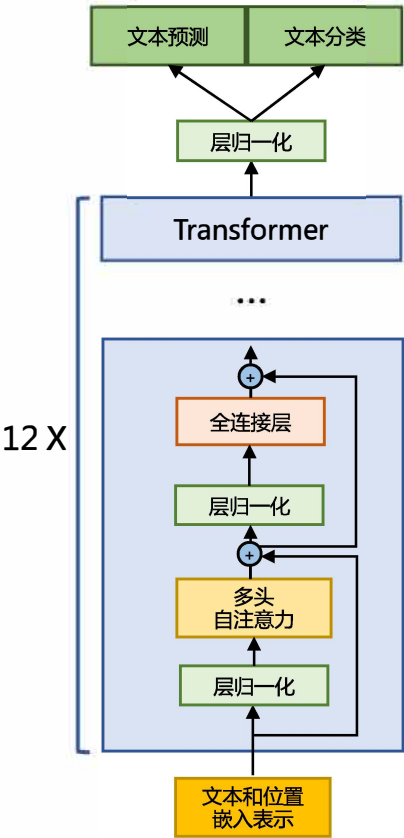


图 2.4 GPT-2 模型结构

接下来，将分别介绍 RMSNorm 归一化函数、SwiGLU 激活函数和旋转位置嵌入 (RoPE) 的具体内容和实现。

1. RMSNorm 归一化函数

为了使得模型训练过程更加稳定，GPT-2 相较于 GPT 就引入了前置层归一化方法，将第一个层归一化移动到多头自注意力层之前，第二个层归一化也移动到了全连接层之前，同时残差连接的位置也调整到了多头自注意力层与全连接层之后。层归一化中也采用了 RMSNorm 归一化函数^[49]。针对输入向量 $\mathbf{a}$ RMSNorm 函数计算公式如下：

$$RMS(\mathbf{a}) = \sqrt{\frac{1}{n} \sum_{i=1}^n \mathbf{a}_i^2} \quad (2.14)$$

$$\bar{\mathbf{a}}_i = \frac{\mathbf{a}_i}{RMS(\mathbf{a})} \quad (2.15)$$

此外，RMSNorm 还可以引入可学习的缩放因子 g_i 和偏移参数 b_i ，从而得到 $\bar{\mathbf{a}}_i = \frac{\mathbf{a}_i}{RMS(\mathbf{a})} g_i + b_i$ 。RMSNorm 在 HuggingFace Transformer 库中代码实现如下所示：

```
class LlamaRMSNorm(nn.Module):
    def __init__(self, hidden_size, eps=1e-6):
        """
        LlamaRMSNorm is equivalent to T5LayerNorm
        """
        super().__init__()
        self.weight = nn.Parameter(torch.ones(hidden_size))
        self.variance_epsilon = eps # eps 防止取倒数之后分母为 0

    def forward(self, hidden_states):
        input_dtype = hidden_states.dtype
        variance = hidden_states.to(torch.float32).pow(2).mean(-1, keepdim=True)
        hidden_states = hidden_states * torch.rsqrt(variance + self.variance_epsilon)
        # weight 是末尾乘的可训练参数，即  $g_i$ 
        return (self.weight * hidden_states).to(input_dtype)
```

2. SwiGLU 激活函数

SwiGLU^[50] 激活函数是 Shazeer 在文献 [50] 中提出，并在 PaLM^[14] 等模中进行了广泛应用，并且取得了不错的效果，相较于 ReLU 函数在大部分评测中都有不少提升。在 LLaMA 中全连接层使用带有 SwiGLU 激活函数的 FFN（Position-wise Feed-Forward Network）的计算公式如下：

$$\text{FFN}_{\text{SwiGLU}}(\mathbf{x}, \mathbf{W}, \mathbf{V}, \mathbf{W}_2) = \text{SwiGLU}(\mathbf{x}, \mathbf{W}, \mathbf{V}) \mathbf{W}_2 \quad (2.16)$$

$$\text{SwiGLU}(\mathbf{x}, \mathbf{W}, \mathbf{V}) = \text{Swish}_\beta(\mathbf{x} \mathbf{W}) \otimes \mathbf{x} \mathbf{V} \quad (2.17)$$

$$\text{Swish}_\beta(\mathbf{x}) = \mathbf{x} \sigma(\beta \mathbf{x}) \quad (2.18)$$

其中， $\sigma(x)$ 是 Sigmoid 函数。图2.5给出了 Swish 激活函数在参数 β 不同取值下的形状。可以看到当 β 趋近于 0 时，Swish 函数趋近于线性函数 $y = x$ ，当 β 趋近于无穷大时，Swish 函数趋近

于 ReLU 函数， β 取值为 1 时，Swish 函数是光滑且非单调。在 HuggingFace 的 Transformer 库中 Swish₁ 函数使用 silu 函数^[51] 代替。

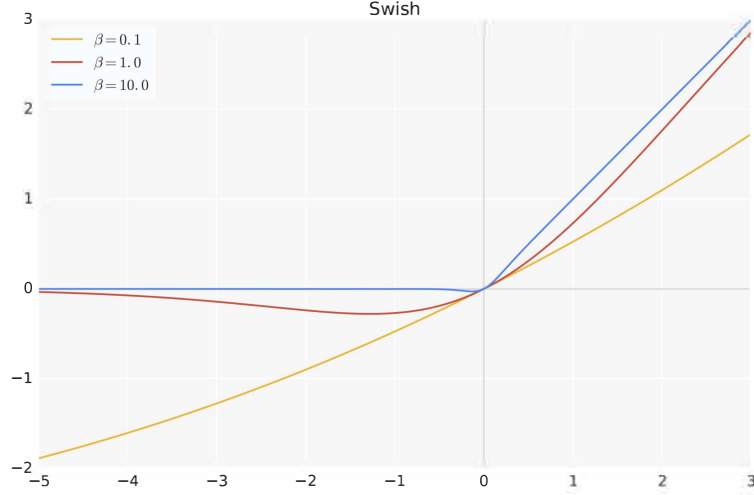


图 2.5 Swish 激活函数在参数 β 不同取值下的形状

3. 旋转位置嵌入 (RoPE)

在位置编码上，使用旋转位置嵌入 (Rotary Positional Embeddings, RoPE)^[52] 代替原有的绝对位置编码。RoPE 借助了复数的思想，出发点是通过绝对位置编码的方式实现相对位置编码。其目标是通过下述运算来给 $\mathbf{q}$ ， $\mathbf{k}$ 添加绝对位置信息：

$$\tilde{\mathbf{q}}_m = f(\mathbf{q}, m), \tilde{\mathbf{k}}_n = f(\mathbf{k}, n) \quad (2.19)$$

经过上述操作后， $\tilde{\mathbf{q}}_m$ 和 $\tilde{\mathbf{k}}_n$ 就带有位置 m 和 n 的绝对位置信息。

详细的证明和求解过程可以参考文献 [52]，最终可以得到二维情况下用复数表示的 RoPE：

$$f(\mathbf{q}, m) = R_f(\mathbf{q}, m)e^{i\Theta_f(\mathbf{q}, m)} = \|\mathbf{q}\|e^{i(\Theta(\mathbf{q})+m\theta)} = \mathbf{q}e^{im\theta} \quad (2.20)$$

根据复数乘法的几何意义，上述变换实际上是对应向量旋转，所以位置向量称为“旋转式位置编码”。还可以使用矩阵形式表示：

$$f(\mathbf{q}, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} \mathbf{q}_0 \\ \mathbf{q}_1 \end{pmatrix} \quad (2.21)$$

根据内积满足线性叠加的性质，任意偶数维的 RoPE，都可以表示为二维情形的拼接，即：

$$f(\mathbf{q}, m) = \underbrace{\begin{pmatrix} \cos m\theta_0 & -\sin m\theta_0 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_0 & \cos m\theta_0 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_1 & -\sin m\theta_1 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_1 & \cos m\theta_1 & \cdots & 0 & 0 \\ \cdots & \cdots & \cdots & \cdots & \ddots & \cdots & \cdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2-1} & -\sin m\theta_{d/2-1} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2-1} & \cos m\theta_{d/2-1} \end{pmatrix}}_{\mathbf{R}_d} \begin{pmatrix} \mathbf{q}_0 \\ \mathbf{q}_1 \\ \mathbf{q}_2 \\ \mathbf{q}_3 \\ \cdots \\ \mathbf{q}_{d-2} \\ \mathbf{q}_{d-1} \end{pmatrix} \quad (2.22)$$

由于上述矩阵 $\mathbf{R}_n$ 具有稀疏性，因此可以使用逐位相乘 $\otimes$ 操作进一步加快计算速度。RoPE 在 HuggingFace Transformer 库中代码实现如下所示：

```
class LlamaRotaryEmbedding(torch.nn.Module):
    def __init__(self, dim, max_position_embeddings=2048, base=10000, device=None):
        super().__init__()
        inv_freq = 1.0 / (base ** (torch.arange(0, dim, 2).float().to(device) / dim))
        self.register_buffer("inv_freq", inv_freq)

        # Build here to make `torch.jit.trace` work.
        self.max_seq_len_cached = max_position_embeddings
        t = torch.arange(self.max_seq_len_cached, device=self.inv_freq.device,
                        dtype=self.inv_freq.dtype)
        freqs = torch.einsum("i,j->ij", t, self.inv_freq)
        # Different from paper, but it uses a different permutation
        # in order to obtain the same calculation
        emb = torch.cat((freqs, freqs), dim=-1)
        dtype = torch.get_default_dtype()
        self.register_buffer("cos_cached", emb.cos()[None, None, :, :].to(dtype), persistent=False)
        self.register_buffer("sin_cached", emb.sin()[None, None, :, :].to(dtype), persistent=False)

    def forward(self, x, seq_len=None):
        # x: [bs, num_attention_heads, seq_len, head_size]
        # This `if` block is unlikely to be run after we build sin/cos in `__init__`.
        # Keep the logic here just in case.
        if seq_len > self.max_seq_len_cached:
            self.max_seq_len_cached = seq_len
            t = torch.arange(self.max_seq_len_cached, device=x.device, dtype=self.inv_freq.dtype)
            freqs = torch.einsum("i,j->ij", t, self.inv_freq)
            # Different from paper, but it uses a different permutation
            # in order to obtain the same calculation
            emb = torch.cat((freqs, freqs), dim=-1).to(x.device)
            self.register_buffer("cos_cached", emb.cos()[None, None, :, :].to(x.dtype),
                                persistent=False)
            self.register_buffer("sin_cached", emb.sin()[None, None, :, :].to(x.dtype),
                                persistent=False)
```

```

        return (
            self.cos_cached[:, :, :seq_len, ...].to(dtype=x.dtype),
            self.sin_cached[:, :, :seq_len, ...].to(dtype=x.dtype),
        )

def rotate_half(x):
    """Rotates half the hidden dims of the input."""
    x1 = x[..., : x.shape[-1] // 2]
    x2 = x[..., x.shape[-1] // 2 :]
    return torch.cat((-x2, x1), dim=-1)

def apply_rotary_pos_emb(q, k, cos, sin, position_ids):
    # The first two dimensions of cos and sin are always 1, so we can `squeeze` them.
    cos = cos.squeeze(1).squeeze(0) # [seq_len, dim]
    sin = sin.squeeze(1).squeeze(0) # [seq_len, dim]
    cos = cos[position_ids].unsqueeze(1) # [bs, 1, seq_len, dim]
    sin = sin[position_ids].unsqueeze(1) # [bs, 1, seq_len, dim]
    q_embed = (q * cos) + (rotate_half(q) * sin)
    k_embed = (k * cos) + (rotate_half(k) * sin)
    return q_embed, k_embed

```

4. 模型整体框架

基于上述模型和网络结构可以实现解码器层，根据自回归方式利用训练语料进行模型的过程与本章第 2.3.4 节介绍的过程基本一致。不同规模 LLaMA 模型所使用的具体超参数如表 2.1 所示。但是由于大语言模型的参数量非常大，并且需要大量的数据进行训练，因此仅利用单个 GPU 很难完成训练，需要依赖分布式模型训练框架（本书第 4 章将详细介绍相关内容）。

表 2.1 LLaMA 不同模型规模下的具体超参数细节^[37]

参数规模	层数	自注意力头数	嵌入表示维度	学习率	全局批次大小	训练 Token 数
6.7B	32	32	4096	3.0e-4	400 万	1.0 万亿
13.0B	40	40	5120	3.0e-4	400 万	1.0 万亿
32.5B	60	52	6656	1.5e-4	400 万	1.4 万亿
65.2B	80	64	8192	1.5e-4	400 万	1.4 万亿

HuggingFace Transformer 库中 LLaMA 解码器整体实现代码实现如下所示：

```

class LlamaDecoderLayer(nn.Module):
    def __init__(self, config: LlamaConfig):
        super().__init__()
        self.hidden_size = config.hidden_size
        self.self_attn = LlamaAttention(config=config)
        self.mlp = LlamaMLP(

```

```

        hidden_size=self.hidden_size,
        intermediate_size=config.intermediate_size,
        hidden_act=config.hidden_act,
    )
    self.input_layernorm = LlamaRMSNorm(config.hidden_size, eps=config.rms_norm_eps)
    self.post_attention_layernorm = LlamaRMSNorm(config.hidden_size, eps=config.rms_norm_eps)

def forward(
    self,
    hidden_states: torch.Tensor,
    attention_mask: Optional[torch.Tensor] = None,
    position_ids: Optional[torch.LongTensor] = None,
    past_key_value: Optional[Tuple[torch.Tensor]] = None,
    output_attentions: Optional[bool] = False,
    use_cache: Optional[bool] = False,
) -> Tuple[torch.FloatTensor, Optional[Tuple[torch.FloatTensor, torch.FloatTensor]]]:

    residual = hidden_states
    hidden_states = self.input_layernorm(hidden_states)

    # Self Attention
    hidden_states, self_attn_weights, present_key_value = self.self_attn(
        hidden_states=hidden_states,
        attention_mask=attention_mask,
        position_ids=position_ids,
        past_key_value=past_key_value,
        output_attentions=output_attentions,
        use_cache=use_cache,
    )
    hidden_states = residual + hidden_states

    # Fully Connected
    residual = hidden_states
    hidden_states = self.post_attention_layernorm(hidden_states)
    hidden_states = self.mlp(hidden_states)
    hidden_states = residual + hidden_states

    outputs = (hidden_states,)

    if output_attentions:
        outputs += (self_attn_weights,)

    if use_cache:
        outputs += (present_key_value,)

    return outputs

```

2.3.2 注意力机制优化

在 Transformer 结构中，自注意力机制的时间和存储复杂度与序列的长度呈平方的关系，因此占用了大量的计算设备内存和并消耗大量计算资源。因此，如何优化自注意力机制的时空复杂度、增强计算效率是大语言模型需要面临的重要问题。一些研究从近似注意力出发，旨在减少注意力

计算和内存需求，提出了包括稀疏近似、低秩近似等方法。此外，也有一些研究从计算加速设备本身的特性出发，研究如何更好利用硬件特性对 Transformer 中注意力层进行高效计算。本节中，将分别介绍上述两类方法。

1. 稀疏注意力机制

通过对一些训练好的 Transformer 模型中的注意力矩阵进行分析发现，其中很多通常是稀疏的，因此可以通过限制 Query-Key 对的数量来减少计算复杂度。这类方法就称为稀疏注意力（Sparse Attention）机制。可以将稀疏化方法进一步分成两类：基于位置信息和基于内容。

基于位置的稀疏注意力机制的基本类型如图2.6所示，主要包含如下五种类型：（1）全局注意力（Global Attention）：为了增强模型建模长距离依赖关系，可以加入一些全局节点；（2）带状注意力（Band Attention）：大部分数据都带有局部性，限制 Query 只与相邻的几个节点进行交互；（3）膨胀注意力（Dilated Attention）；与 CNN 中的 Dilated Conv 类似，通过增加空隙以获取更大的感受野；（4）随机注意力（Random Attention）：通过随机采样，提升非局部的交互；（5）局部块注意力（Block Local Attention）：使用多个不重叠的块（Block）来限制信息交互。

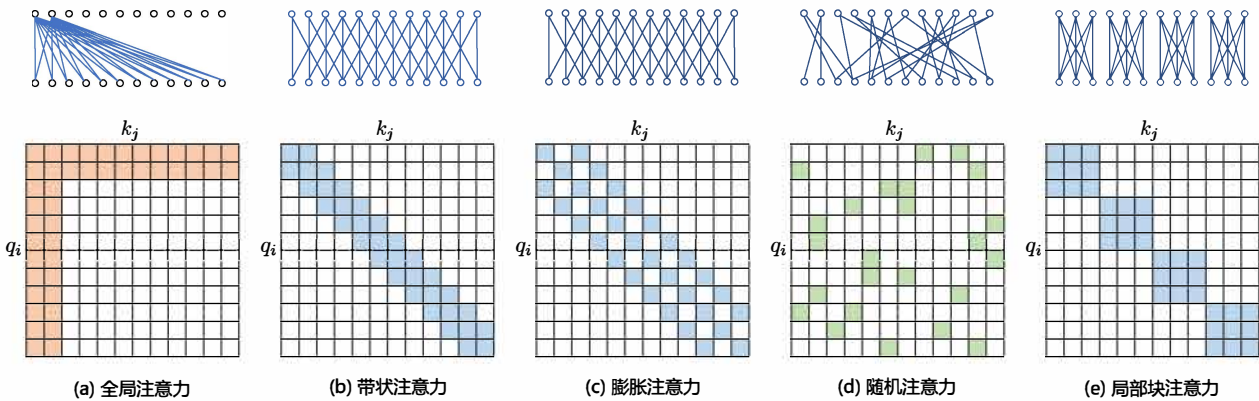


图 2.6 五种基于位置的稀疏注意力基本类型^[53]

现有的稀疏注意力机制，通常是基于上述五种基本基于位置的稀疏注意力机制的复合模式，图2.7给出了一些典型的稀疏注意力模型。Star-Transformer^[54] 使用带状注意力和全局注意力的组合。具体来说，Star-Transformer 只包括一个全局注意力节点和宽度为 3 的带状注意力，其中任意两个非相邻节点通过一个共享的全局注意力连接，而相邻节点则直接相连。Longformer^[55] 使用带状注意力和内部全局节点注意力（Internal Global-node Attention）的组合。此外，Longformer 还将上层中的一些带状注意力头部替换为具有扩张窗口的注意力，在增加感受野同时并不增加计算量。Extended Transformer Construction (ETC)^[56] 利用带状注意力和外部全局节点注意力（External Global-node Attention）的组合。ETC 稀疏注意力还包括一种掩码机制来处理结构化输入，并采用对比预测编码（Contrastive Predictive Coding, CPC）^[57] 进行预训练。BigBird^[58] 使用带状和全局

注意力，还使用额外的随机注意力来近似全连接注意力，此外还揭示了稀疏编码器和稀疏解码器的使用可以模拟任何图灵机，这也在一定程度上解释了，为什么稀疏注意力模型可以取得较好的结果原因。

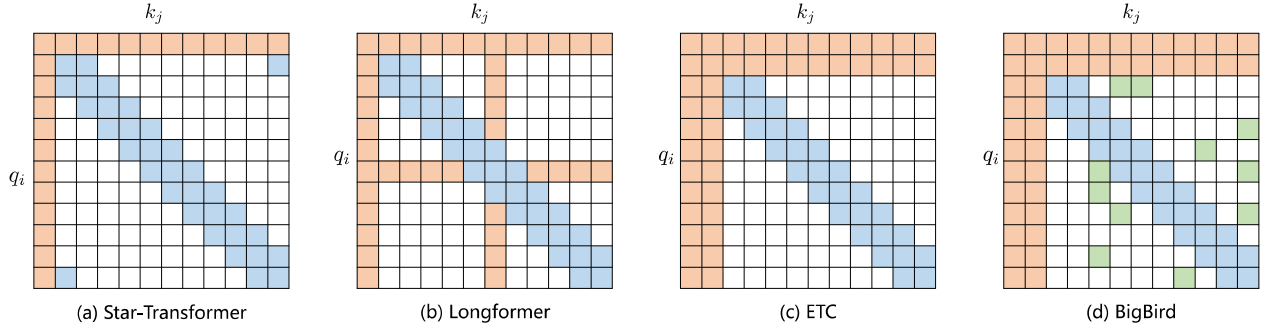


图 2.7 基于位置复合稀疏注意力类型^[53]

基于内容的稀疏注意力是根据输入数据来创建稀疏注意力，其中一种很简单的方法是选择和给定查询（Query）有很高相似度的键（Key）。Routing Transformer^[59] 采用 K-means 聚类方法，针对 $\text{Query}\{q_i\}_{i=1}^T$ 和 $\text{Key}\{k_i\}_{i=1}^T$ 一起进行聚类，类中心向量集合为 $\{\mu_i\}_{i=1}^k$ ，其中 k 是类中心个数。每个 Query 只与其处在相同簇（Cluster）下的 Key 进行交互。中心向量采用滑动平均的方法进行更新：

$$\tilde{\mu} \leftarrow \lambda \tilde{\mu} + (1 - \lambda) \left(\sum_{i: \mu(q_i) = \mu} q_i + \sum_{j: \mu(k_j) = \mu} k_j \right) \quad (2.23)$$

$$c_\mu \leftarrow \lambda c_\mu + (1 - \lambda) |\mu| \quad (2.24)$$

$$\mu \leftarrow \frac{\tilde{\mu}}{c_\mu} \quad (2.25)$$

其中 $|\mu|$ 表示在簇 μ 中向量的数量。

Reformer^[60] 则采用局部敏感哈希（Local-Sensitive Hashing, LSH）方法来为每个 Query 选择 Key-Value 对。其主要思想使用 LSH 函数将 Query 和 Key 进行哈希计算，将它们划分到多个桶内。提升在同一个桶内的 Query 和 Key 参与交互的概率。假设 b 是桶的个数，给定一个大小为 $[D_k, b/2]$ 随机矩阵 R ，LSH 函数定义为：

$$h(x) = \arg \max([xR; -xR]) \quad (2.26)$$

如果 $hq_i = hk_j$ 时， q_i 才可以与相应的 Key-Value 对进行交互。

2. FlashAttention

NVIDIA GPU 中的内存（显存）按照它们物理上是在 GPU 芯片内部还是板卡 RAM 存储芯片上，决定了它们的速度、大小以及访问限制。GPU 显存分为全局内存（Global memory）、本地内存（Local memory）、共享内存（Shared memory, SRAM）、寄存器内存（Register memory）、常量内存（Constant memory）、纹理内存（Texture memory）等六大类。图2.8给出了 NVIDIA GPU 内存的整体结构。其中全局内存、本地内存、共享内存和寄存器内存具有读写能力。全局内存和本地内存使用的高带宽显存（High Bandwidth Memory, HBM）位于板卡 RAM 存储芯片上，该部分内存容量很大。全局内存是所有线程都可以访问，而本地内存则只能当前线程访问。NVIDIA H100 中全局内存有 80GB 空间，其访问速度虽然可以达到 3.35TB/s，但是如果全部线程同时访问全局内存时，其平均带宽仍然很低。共享内存和寄存器位于 GPU 芯片上，因此容量很小，并且共享内存只有在同一个 GPU 线程块（Thread Block）内的线程才可以共享访问，而寄存器仅限于同一个线程内部才能访问。NVIDIA H100 中每个 GPU 线程块在流式多处理器（Stream Multi-processor, SM）可以使用的共享存储容量仅有 228KB，但是其速度非常快，远高于全局内存的访问速度。

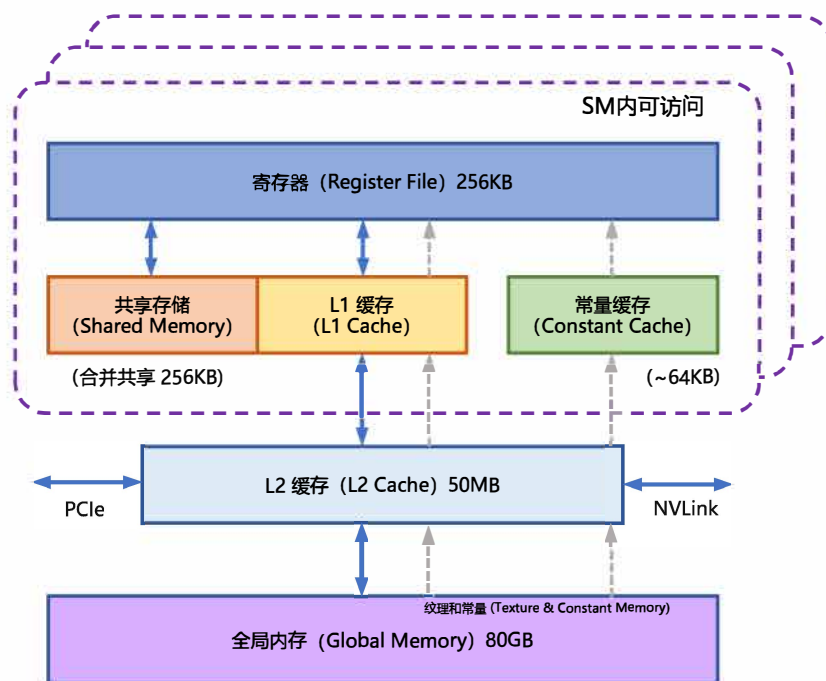


图 2.8 NVIDIA GPU 的整体内存结构图

在本章第 2.2 节中介绍自注意力机制的原理，在 GPU 中进行计算时，传统的方法还需要引入

两个中间矩阵 S 和 P 并存储到全局内存中。具体计算过程如下：

$$S = Q \times K, P = \text{Softmax}(S), O = P \times V \quad (2.27)$$

按照上述计算过程，需要首先从全局内存中读取矩阵 Q 和 K ，并将计算好的矩阵 S 再写入全局内存，之后再从全局内存中获取矩阵 S ，计算 Softmax 得到矩阵 P ，再写入全局内容，之后读取矩阵 P 和矩阵 V ，计算得到矩阵 O 。这样的过程会极大占用显存的带宽。在自注意力机制中，计算速度比内存速度快得多，因此计算效率越来越多地受到全局内存访问的瓶颈。

FlashAttention^[61] 就是通过利用 GPU 硬件中的特殊设计，针对全局内存和共享存储的 I/O 速度的不同，尽可能的避免 HBM 中读取或写入注意力矩阵。FlashAttention 目标是尽可能高效地使用 SRAM 来加快计算速度，避免从全局内存中读取和写入注意力矩阵。达成该目标需要能做到在不访问整个输入的情况下计算 Softmax 函数，并且后向传播中不能存储中间注意力矩阵。标准 Attention 算法中，Softmax 计算按行进行，即在与 V 做矩阵乘法之前，需要将 Q 、 K 的各个分块完成一整行的计算。在得到 Softmax 的结果后，再与矩阵 V 分块做矩阵乘。而在 FlashAttention 中，将输入分割成块，并在输入块上进行多次传递，从而以增量方式执行 Softmax 计算。

自注意力算法的标准实现将计算过程中的矩阵 S 、 P 写入全局内存中，而这些中间矩阵的大小与输入的序列长度有关且为二次型。因此，FlashAttention 就提出了不使用中间注意力矩阵，通过存储归一化因子来减少全局内存的消耗。FlashAttention 算法并没有将 S 、 P 整体写入全局内存，而是通过分块写入，存储前向传递的 Softmax 归一化因子，在后向传播中快速重新计算片上注意力，这比从全局内容中读取中间注意力矩阵的标准方法更快。由于大幅度减少了全局内存的访问量，即使重新计算导致 FLOPs 增加，但其运行速度更快并且使用更少的内存。具体算法如代码 2.1 所示，其中内循环和外循环所对应的计算可以参考图 2.9。

PyTorch 2.0 中已经可以支持 FlashAttention，使用 “`torch.backends.cuda.enable_flash_sdp()`” 启用或者关闭 FlashAttention 的使用。

3. 多查询注意力

多查询注意力 (Multi Query Attention)^[62] 是多头注意力的一种变体。其主要区别在于，在多查询注意力中不同的注意力头共享一个键和值的集合，每个头只单独保留了一份查询参数。因此键和值的矩阵仅有一份，这大幅度减少了显存占用，使其更高效。由于多查询注意力改变了注意力机制的结构，因此模型通常需要从训练开始就支持多查询注意力。文献 [63] 的研究结果表明，可以通过对已经训练好的模型进行微调来添加多查询注意力支持，仅需要约 5% 的原始训练数据量就可以达到不错的效果。包括 Falcon^[64]、SantaCoder^[65]、StarCoder^[66] 等在内很多模型都采用了多查询注意力机制。

以 LLM Foundry 为例，多查询注意力实现代码如下：

代码 2.1: FlashAttention 算法

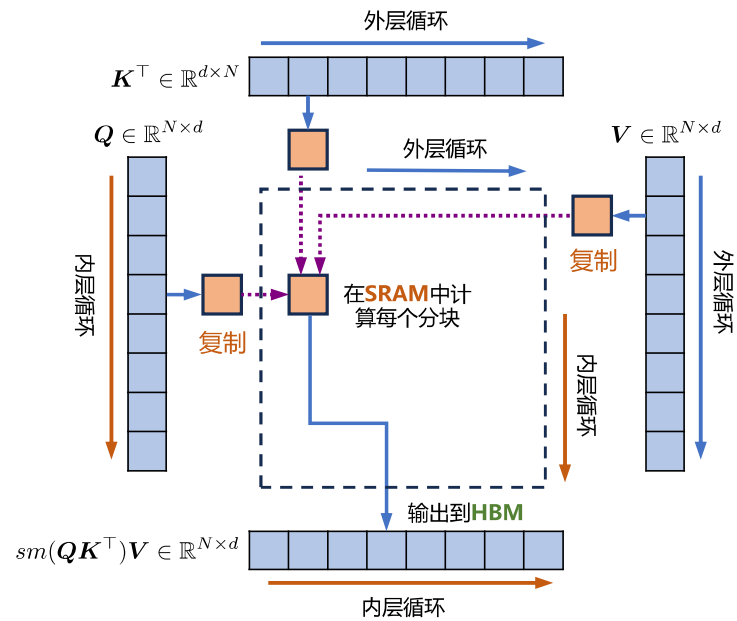
输入: $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ 位于高速显存 (HBM) 中, GPU 芯片中的 SRAM 大小为 M
输出: $\mathbf{O}$
 $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$ // 设置块大小 (*block size*)
在 HBM 中初始化 $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\mathbf{l} = (0)_N \in \mathbb{R}^N$, $\mathbf{m} = (-\infty)_N \in \mathbb{R}^N$
将矩阵 $\mathbf{Q}$ 切分成 $T_r = \lceil \frac{M}{B_r} \rceil$ 块 $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$, $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$
将矩阵 $\mathbf{K}$ 切分成 $T_c = \lceil \frac{M}{B_c} \rceil$ 块 $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$, $\mathbf{K}_i \in \mathbb{R}^{B_c \times d}$
将矩阵 $\mathbf{V}$ 切分成 T_c 块 $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, $\mathbf{V}_i \in \mathbb{R}^{B_c \times d}$
将矩阵 $\mathbf{O}$ 切分成 T_r 块 $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$, $\mathbf{O}_i \in \mathbb{R}^{B_r \times d}$
将 $\mathbf{l}$ 切分成 T_r 块 $\mathbf{l}_1, \dots, \mathbf{l}_{T_r}$, $\mathbf{l}_i \in \mathbb{R}^{B_r}$
将 $\mathbf{m}$ 切分成 T_r 块 $\mathbf{m}_1, \dots, \mathbf{m}_{T_r}$, $\mathbf{m}_i \in \mathbb{R}^{B_r}$
for $j = 1$ **to** T_c **do**
 将 $\mathbf{K}_j$ 和 $\mathbf{V}_j$ 从 HBM 中读入芯片存储 SRAM
 for $i = 1$ **to** T_r **do**
 计算 $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$
 计算 $\tilde{\mathbf{m}}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{\mathbf{m}}_{ij}) \in \mathbb{R}^{B_r \times B_c}$
 计算 $\tilde{\mathbf{l}}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$
 计算 $\mathbf{m}_i^{new} = \max(\mathbf{m}_i, \tilde{\mathbf{m}}_{ij}) \in \mathbb{R}^{B_r}$, $\mathbf{l}_i^{new} = e^{\mathbf{m}_i - \mathbf{m}_i^{new}} \mathbf{l}_i + e^{\tilde{\mathbf{m}}_{ij} - \mathbf{m}_i^{new}} \tilde{\mathbf{l}}_{ij} \in \mathbb{R}^{B_r}$
 将 $\mathbf{O} \leftarrow \text{diag}(\mathbf{l}_i^{new})^{-1} (\text{diag}(\mathbf{l}_i) e^{\mathbf{m}_i - \mathbf{m}_i^{new}} \mathbf{O}_i + e^{\tilde{\mathbf{m}}_{ij} - \mathbf{m}_i^{new}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ 写回 HBM 中
 将 $\mathbf{l}_i \leftarrow \mathbf{l}_i^{new}$ 和 $\mathbf{m}_i \leftarrow \mathbf{m}_i^{new}$ 写回 HBM 中
 end
end
return $\mathbf{O}$

```
class MultiQueryAttention(nn.Module):
    """Multi-Query self attention.

    Using torch or triton attention implemetation enables user to also use
    additive bias.
    """

    def __init__(
        self,
        d_model: int,
        n_heads: int,
        device: Optional[str] = None,
    ):
        super().__init__()

        self.d_model = d_model
        self.n_heads = n_heads
        self.head_dim = d_model // n_heads
```

图 2.9 FlashAttention 计算流程图^[61]

```

self.Wqkv = nn.Linear(                                # Multi-Query Attention 创建
    d_model,
    d_model + 2 * self.head_dim,                       # 只创建 查询 的头向量, 所以只有 1 个 d_model
    device=device,                                       # 而 键 和 值 则共享各自的一个 head_dim 的向量
)

self.attn_fn = scaled_multihead_dot_product_attention
self.out_proj = nn.Linear(
    self.d_model,
    self.d_model,
    device=device
)
self.out_proj._is_residual = True # type: ignore

def forward(
    self,
    x,
):
    qkv = self.Wqkv(x)                                  # (1, 512, 960)

    query, key, value = qkv.split(                       # query -> (1, 512, 768)
        [self.d_model, self.head_dim, self.head_dim],  # key   -> (1, 512, 96)
        dim=2                                           # value -> (1, 512, 96)
    )

    context, attn_weights, past_key_value = self.attn_fn(
        query,
        key,
        value,
    )

```

```

        self.n_heads,
        multiquery=True,
    )

    return self.out_proj(context), attn_weights, past_key_value

```

与 LLM Foundry 中实现的多头自注意力代码相对比，其区别仅在于建立 Wqkv 层上：

```

# Multi Head Attention
self.Wqkv = nn.Linear(
    self.d_model,
    3 * self.d_model,
    device=device
)

query, key, value = qkv.chunk(
    3,
    dim=2
)

# Multi Query Attention
self.Wqkv = nn.Linear(
    d_model,
    d_model + 2 * self.head_dim,
    device=device,
)

query, key, value = qkv.split(
    [self.d_model, self.head_dim, self.head_dim],
    dim=2
)

```

Multi-Head Attention 的创建方法
*# 查询、键和值 3 个矩阵，所以是 3 * d_model*
每个 tensor 都是 (1, 512, 768)

Multi-Query Attention 的创建方法
只创建查询的头向量，所以是 1 d_model*
而键和值不再具备单独的头向量
query -> (1, 512, 768)
key -> (1, 512, 96)
value -> (1, 512, 96)